

Automatyczne wykrywanie i analiza aktywności transakcyjnej w blockchainie po incydentach cyberbezpieczeństwa

#	Zadanie	Status	Co mam / czego brakuje
1	Model zautomatyzowanego śledzenia transakcji	Zrobione	Graf BFS (1–3 hopy) z terminacją na znanych endpointach; heurystyki (Tornado/CEX/bridge/peel) na całym grafie. Brak: śledzenie wywołań smart kontraktów.
2	Prototyp z wykorzystaniem Arkham API	Zrobione	Klient Arkham (batch lookup, kategorie), etykiety w UI, dyskowy cache surowych odpowiedzi API. Działa end-to-end.
3	Badania eksperymentalne na rzeczywistych przypadkach	Zrobione	3 case studies, okno incydentu, eksperyment z zapisem wyników (JSON/MD/CSV); run hops=3 wykonany.
4	Ocena efektywności	zrobione	4 metryki (Address Recall, Heuristic Precision/Recall, CEX Coverage) vs ground truth + breakdown;

Architektura

Aplikacja składa się z dwóch części:

- Backend (Python 3.11) - warstwa logiki i danych. Pobiera transakcje z **Etherscan** API (sieć Ethereum: transakcje ETH oraz transfery tokenów ERC-20), a etykiety adresów z **Arkham Intelligence**. Na tej podstawie buduje graf przepływu środków, uruchamia heurystyki wykrywające pranie i liczy metryki skuteczności.
- Frontend (Next.js / React / TypeScript) - interfejs webowy prezentujący wyniki: graf, alerty, transakcje, etykiety i metryki, oraz tryb porównujący kilka przypadków naraz.

Przebieg analizy:

użytkownik podaje adres startowy → backend przechodzi graf metodą BFS przez 1–3 poziomy, dla każdego adresu pobierając jego transakcje; znane punkty końcowe (miksery, giełdy, mosty) są traktowane jako naturalna terminacja śledzenia → na pełnym grafie uruchamiane są heurystyki (Tornado Cash, giełdy CEX, mosty cross-chain, peel chain) → wynik jest porównywany z udokumentowanym ground truth danego incydentu i oceniany ilościowo.

Dla powtarzalności i oszczędności limitów API surowe odpowiedzi z Etherscan/Arkham są cache'owane na dysku - ponowne uruchomienie tej samej analizy nie odpytuje API/

Widok główny - analiza adresu atakującego Ronin Bridge

Po wpisaniu adresu i uruchomieniu analizy aplikacja automatycznie identyfikuje adres źródłowy (etykieta z Arkham i prezentuje podsumowanie: liczbę transakcji, etykiet oraz alertów w podziale na poziomy ważności (critical/warning/info). Na dole najważniejsze sygnały wykryte przez heurystyki (kontakt z CEX, peel chain).

Analiza w oknie incydentu (7 dni po ataku), głębokość 2 hopy.

The screenshot displays the 'Forensics Blockchain Tracer' application interface. At the top, there are filters for 'Ronin Bridge (\$625M, 2022)', 'Euler Finance (\$197M, 2023)', and 'Nomad Bridge (\$190M, 2022)'. A search bar contains the address '0x098B716B8Aaf21512996dc57E80615e2383E2f96'. To the right of the search bar are 'HOPS' buttons (1, 2, 3) with '2' selected, a 'Trace' button, and a green button labeled 'Eksperyment 3 case'y'. Below the search bar are buttons for 'Pobierz na żywo (pomiń cache)' and 'Wyczyść cache'. A status message reads: 'Lazarus Group, OFAC sanctioned. Laundering: Tornado Cash + bridges. 2 hops ≈ 10-30 sek'. A checkbox labeled 'Okno incydentu' is checked, with details: '2022-03-23 · bloki 14,442,835-14,492,835 · ~7 dni post-attack'. A navigation bar includes 'Overview' (highlighted), 'Graph', 'Alerts 8', 'Transactions 50', 'Labels 77', and 'Metrics 1'. The main content area shows the 'ROOT ADDRESS' as 'Ronin Bridge Exploiter (OFAC Sanctioned)' with a 'HACKER' tag and the address '0x098b716b8aaf21512996dc57eb0615e2383e2f96'. Below this are five summary boxes: 'TRANSAKCJE 50', 'ETYKIETY 77', 'CRITICAL 0', 'WARNING 8', and 'INFO 0'. The 'NAJWAŻNIEJSZE ALERTY' section lists three warnings: 'Peel chain candidate: 0x098b...2f96 (15 peels + 1 change, CASH)', 'Kontakt z CEX: Binance 16 (przez 0xaf0c...a59c) (1 tx, USDT)', and 'Peel chain candidate: 0x1a2a...54f2 (2 peels + 1 change, USDC)'. A small 'N' icon is visible on the left side of the alerts section.

Graf przepływu środków

Graf pokazuje przepływ środków od adresu atakującego (niebieski, środek) przez adresy pośrednie do punktów końcowych. Kolory oznaczają kategorie: czerwony - hacker/Lazarus, zielony - giełdy CEX (Binance, Coinbase, FTX Deposit), pomarańczowy - mosty cross-chain.

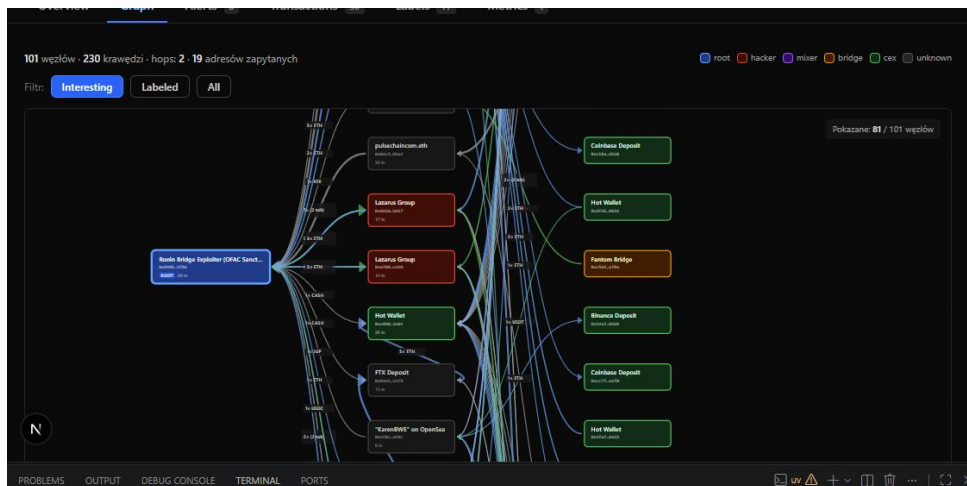


Tabela transakcji

Pełna lista transakcji śledzonego adresu z etykietami stron (HACKER / LABELED). Widok ten ujawnia również praktyczne ograniczenie analizy on-chain: publiczne adresy atakujących są zasypywane spam-tokenami, które wprowadzają szum do danych.

Pobierz na żywo (pomiń cache) Wyczyść cache

Lazarus Group, OFAC sanctioned, Laundering: Tornado Cash + bridges. 2 hops w 10-30 sek

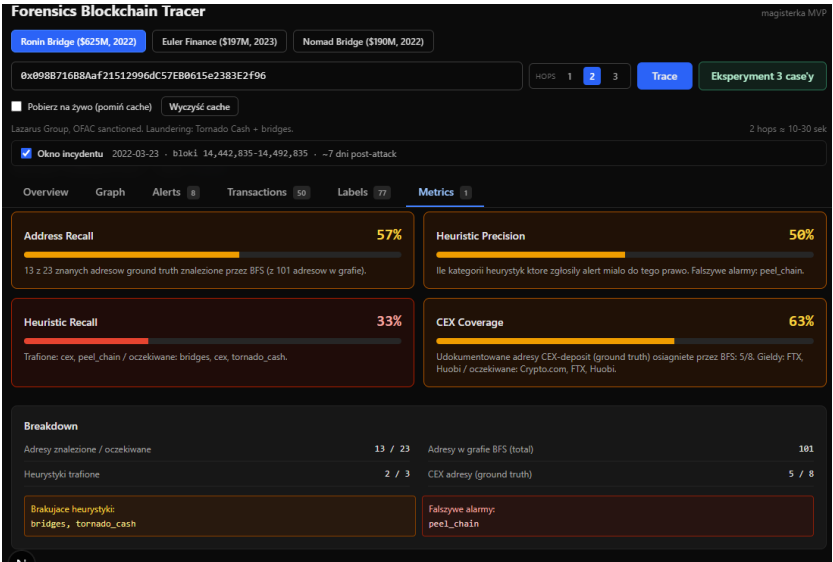
☒ Okno incydentu 2022-03-23 · blok1 14,442,835-14,492,835 · ~7 dni post-attack

Overview Graph Alerts **Transactions 50** Labels 77 Metrics 1

Hash	Block	From	To	Value	Token
0x57a055b97966...	14485762	HACKER Ronin Bridge Exploiter (OFAC Sanctioned)	0x5e7c...f5cd	1	CASH
0x06df0281e0cb...	14485740	HACKER Ronin Bridge Exploiter (OFAC Sanctioned)	0x5e7c...f5cd	1	CASH
0x8cf3b1e14280...	14485735	HACKER Ronin Bridge Exploiter (OFAC Sanctioned)	0x5e7c...f5cd	1	CASH
0xf0e752bcb985...	14485664	HACKER Ronin Bridge Exploiter (OFAC Sanctioned)	0x5e7c...f5cd	1	CASH
0x9ff25aaee0b6...	14483770	HACKER Ronin Bridge Exploiter (OFAC Sanctioned)	0x5e7c...f5cd	1	CASH
0x81da75605d2...	14483604	LABELED pulsechaincom.eth	HACKER Ronin Bridge Exploiter (OFAC Sanctioned)	42	PLS
0xdf5ac680968...	14483604	LABELED pulsechaincom.eth	HACKER Ronin Bridge Exploiter (OFAC Sanctioned)	42	PLS
0xb2e783465e8...	14483602	LABELED pulsechaincom.eth	HACKER Ronin Bridge Exploiter (OFAC Sanctioned)	42	PLS
0x186f9d50d459...	14483602	LABELED pulsechaincom.eth	HACKER Ronin Bridge Exploiter (OFAC Sanctioned)	42	PLS
0x4b3e840c6225...	14483598	LABELED pulsechaincom.eth	HACKER Ronin Bridge Exploiter (OFAC Sanctioned)	42	PLS
0x45ef32f353ae...	14483598	LABELED pulsechaincom.eth	HACKER Ronin Bridge Exploiter (OFAC Sanctioned)	42	PLS

Metryki skuteczności

Ilościowa ocena pojedynczego przypadku względem publicznie udokumentowanego stanu faktycznego: Address Recall 57% (13/23 znanych adresów), Heuristic Precision 50%, Heuristic Recall 33% (trafione cex + peel, brak tornado/bridges w tym oknie/głębokości), CEX Coverage 63%



Eksperyment 3 przypadków

Jednym kliknięciem uruchamiany jest pełny eksperyment na trzech rzeczywistych incydentach (Ronin, Euler, Nomad) z automatycznym porównaniem metryk i zapisem wyników (JSON/MD/CSV). CEX Coverage dla Ronin osiąga 100% (8/8 adresów) przy hops=3, a dla Euler/Nomad oznaczone jest jako N/A (ich ground truth nie zawiera adresów giełdowych brak czego mierzyć, nie błąd).

Forensics Blockchain Tracer

Ronin Bridge (\$625M, 2022) Euler Finance (\$197M, 2023) Nomad Bridge (\$190M, 2022)

0x098B71688Aaf21512996dC57EB0615e2383E2f96 HOPS 1 2 3 Trace Eksperyment 3 case'y

Pobierz na żywo (pomiń cache) Wyczyść cache

Lazarus Group, OFAC sanctioned. Laundering: Tornado Cash + bridges. 3 hops ≈ 1-5 min, sporo zapytan do Etherscan

☒ Okno incydu 2022-03-23 · bloki 14,442,835-14,492,835 · ~7 dni post-attack

3 case studies

Ronin / Euler / Nomad · hops=3 · okno incydu ON

Kopiuuj Markdown Pobierz CSV

Case	Addr. Recall	Heur. Prec.	Heur. Recall	CEX Cov.	Węzły	Alerty	Latency
Ronin Bridge (\$625M, 2022)	70%	67%	67%	100%	475	43	0.1s
Euler Finance (\$197M, 2023)	50%	33%	50%	N/A	321	16	0.1s
Nomad Bridge (\$190M, 2022)	88%	67%	67%	N/A	862	87	0.2s
Średnia	69%	56%	61%	100%	—	—	—

Cache (suma 3 case'ow): etherscan: 664 hit / 0 miss

Funkcja realizuje przeszukiwanie grafu zaczynając od adresu atakującego.

Kolejka przechowuje pary (adres, głębokość); w pętli pobieramy kolejny adres i pomijamy go, jeśli był już odwiedzony. Następnie sprawdzane są dwa warunki

zatrzymania: jeśli adres jest znanym punktem końcowym (mikser, giełda, most),
dodajemy go do grafu.

jeśli osiągnięto maksymalną głębokość hops, również przerywamy. W przeciwnym razie
pobieramy transakcje adresu z Etherscan

```
while queue:
    addr, depth = queue.popleft()
    if addr in visited:
        continue
    visited.add(addr)

    node = nodes.setdefault(addr, GraphNode(address=addr, depth=depth))
    node.depth = min(node.depth, depth)

    if addr != root and addr in endpoints:
        node.is_endpoint = True
        continue
    if depth >= hops:
        continue
    # Pobierz transakcje adresu
    limit = root_max_tx if addr == root else per_hop_max_tx
    sb = start_block if start_block is not None else 0
    eb = end_block if end_block is not None else 99_999_999
    sort_order = "asc" if start_block is not None else "desc"
    try:
        normal, tokens = await asyncio.gather(
            etherscan.get_normal_transactions(
                addr,
                start_block=sb,
                end_block=eb,
                offset=limit // 2 or 10,
                sort=sort_order,
            ),
            etherscan.get_token_transfers(
                addr,
                start_block=sb,
                end_block=eb,
                offset=limit // 2 or 10,
                sort=sort_order,
            ),
        )
```

Heurystyka skanuje wszystkie transakcje z grafu w poszukiwaniu kontaktu z Tornado Cash. Dla każdej transakcji sprawdza obie strony i dwa kierunki: gdy odbiorcą jest pula Tornado → wpłata, gdy nadawcą jest Tornado → wypłata.

Jeśli druga strona nie jest znanym adresem Tornado, transakcja jest pomijana.

„Obserwowany adres” to ten, który faktycznie dotknął tornado, może być adresem głównym lub pośrednim, co pozwala wykryć pranie prowadzone przez łańcuch pośredników.

```
77
78     for tx in transactions:
79         from_addr = tx.from_address.lower()
80         to_addr = (tx.to_address or "").lower()
81
82         for counterparty, observed, direction in (
83             (to_addr, from_addr, "deposit"),
84             (from_addr, to_addr, "withdraw"),
85         ):
86             if not counterparty or counterparty not in db:
87                 continue
88
89             if not observed or observed in db:
90                 continue
91
92             meta = db[counterparty]
93             key = (counterparty, direction, observed)
94             bucket = seen.setdefault(
95                 key,
96                 {
97                     "observed": observed,
98                     "count": 0,
99                     "first_block": tx.block_number,
100                    "last_block": tx.block_number,
101                    "tx_hashes": [],
102                    "total_value": 0.0,
103                    "asset": meta.get("asset") or tx.token_symbol or "?",
104                    "kind": meta.get("kind", "pool"),
105                    "denomination": meta.get("denomination"),
106                },
107             )
108             bucket["count"] += 1
109             bucket["first_block"] = min(bucket["first_block"], tx.block_number)
110             bucket["last_block"] = max(bucket["last_block"], tx.block_number)
111             bucket["tx_hashes"].append(tx.hash)
112             bucket["total_value"] += tx.value_eth
```

Funkcja porównuje wynik analizy z publicznie udokumentowanym stanem faktycznym danego incydentu. Address Recall liczony jest jako część znanych adresów z ground truth, które znalazły się w grafie/transakcjach/etykietach

Heuristic Precision i Recall liczone są na poziomie kategorii tornado / cex / bridge / peel: precision to udział trafnych alertów wśród wszystkich zgłoszonych, a recall to udział oczekiwanych kategorii, które faktycznie wykryto.

```
"""
ground_truth = load_ground_truth(case_name)

# Address recall -----
expected_addr = _extract_expected_addresses(ground_truth)
found_addr = _extract_found_addresses(trace_result)
addr_found = expected_addr & found_addr
address_recall = _safe_div(len(addr_found), len(expected_addr))

# Heuristic precision / recall -----
hit_categories = _alert_categories_hit(trace_result)
expected_categories = _expected_categories(ground_truth)
allowed_categories = _allowed_categories(ground_truth)

heuristics_correctly_hit = hit_categories & allowed_categories
heuristic_precision = _safe_div(
    len(heuristics_correctly_hit),
    len(hit_categories),
)
heuristic_recall = _safe_div(
    len(hit_categories & expected_categories),
    len(expected_categories),
)
```